

AI for Economic Research: Dynamic Models, Language, and Agents

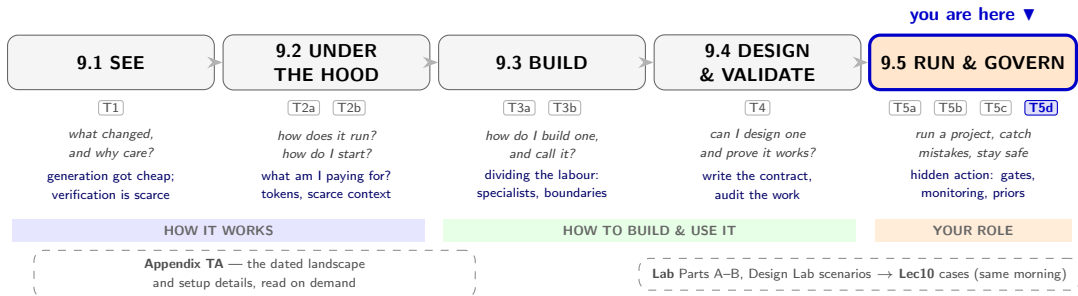
Lecture 9.5d: Governance and Action Plan

Zhigang Feng

2026-09-12 20:56:23

Lecture 9 in One Map: Governance and Action Plan

This deck closes the lecture: the rules that keep this safe, the tests that make it research-grade, and what you do next.



Route: red lines and disclosure → measuring your own agent → limits and judgment → your first thirty days → Lecture 10. The economist's question here: what is never delegated, and what evidence will you accept?

Red Lines

Untrusted inputs, restricted data, and disclosure

Security: Red Lines and Workarounds

Do not put in front of an agent: IRB-protected interviews, linked administrative microdata, personally identifiable information, HIPAA-regulated data, passwords / API keys / access tokens, embargoed or restricted-use datasets.

Operational rule: if you would not paste it into a public collaboration tool, do not paste it into an agent conversation.

Sensitive data does not imply “no agents ever.” Three workarounds:

1. Write the script with the agent, then run it yourself on the restricted data
2. Use synthetic or masked data with the same structure for debugging
3. Use institutionally approved local or private deployments when available

Safe pattern: expose structure and summary statistics, not raw records.

IRB, HIPAA, and GDPR rules apply unchanged to agent-aided work; journal disclosure rules are two frames on.

Treat Everything the Agent Reads as Untrusted

The defining agentic-AI security failure is prompt injection: data the agent merely reads can hijack what it does.

- **OWASP Top-10 for Agentic Applications** (Dec 2025) now codifies these risks.
- Documented exploits: **EchoLeak** (CVE-2025-32711, zero-click M365 Copilot data exfiltration); **ShadowPrompt** in Claude's Chrome extension; Anthropic's **GTG-1002** — the first largely-autonomous cyberattack (Nov 2025).
- **NIST CAISI** red-teaming raised task-hijacking success from **11% to 81%** with adaptive attacks.

Operating rule. Treat any content the agent reads — emails, web pages, retrieved documents, even tool descriptions — as an *untrusted injection channel*. Use least-privilege sandboxing and revocable permissions, exactly as T5b's `settings.json` deny list and the previous frame prescribe. **[solid]**

Disclosure, Replication Packages, and AI_LOG.md

Journals already have rules for this. An agent-assisted paper has to meet them — and the evidence has to exist before submission, not be reconstructed after.

What the AEA journals require

- “Artificial intelligence (AI) software, such as chatbots or other large language models, may not be listed as an author.”
- If AI software was used in preparing the manuscript, “including drafting or editing text,” it must be briefly described at submission; authors remain accountable for checking what it produced
- Papers are published “only if the data and code used in the analysis are clearly and precisely documented” — agent-written code included

AER submission guidelines and AEA journal policies, aeaweb.org, checked 12 Sep 2026. Other

journals differ: read the one you submit to.

What that asks of your workflow

- **Replication package:** the code as run, the data or access instructions, the versions — not the chat
- **AI_LOG.md** (the capstone asks for one): the prompts that mattered, the decisions you made, where the AI was wrong and how you caught it, and an honest split of the work
- **Chain of custody:** date, harness and version, model — next to every result you keep

Disclosure is cheap if you logged as you went, and impossible to fake afterwards. A referee may ask which numbers an agent produced; your log is the answer.

Reproducibility: Chain-of-Custody for Agentic Results

Can an agent reproduce a published computational result? Often only partly — which makes logging non-negotiable.

- **CORE-Bench:** agents reproduce published computational results only **~60% (easy)** / **~21% (hard)**. **[solid]**
- Therefore **log full trajectories:** prompts, tool calls, seeds, *model versions*, *spec versions*, and *dates* — the chain-of-custody for any agent-produced artifact.

Read vendor safety numbers skeptically. Distinguish a vendor's “~1% attack success on a *fixed* suite” from academic findings that *adaptive* attacks still exceed ~85%. A fixed-suite number measures the suite, not the threat model. **[solid]**

Ties to the seven tests below. This is the same discipline (reproducibility, verifiability, auditability) — now with the dates and spec versions the landscape forces you to pin.

Measuring Your Own Agent

A headline number is not a measurement until you know its validity

Benchmarks Break: What SWE-bench Verified Taught

The course's measurement theme, pointed at agents: a headline score is not a measurement until you know what it measures and who could have gamed it.

- **Contamination.** On 23 February 2026 OpenAI stopped reporting SWE-bench Verified: auditing 138 hard tasks, it found 59% had flawed tests, and frontier models could reproduce gold patches verbatim. It recommends SWE-bench Pro instead. [solid]
- **Harness exploits.** In April 2026 Berkeley RDI scored near 100% on eight agent benchmarks — SWE-bench Verified and Pro, Terminal-Bench, WebArena among them — *without solving a single task*, mostly without calling a model. [solid]
- **Never compare across suites.** A Verified score and a Pro score are different problems, difficulty, and exposure — never one axis.

Your own evaluations are benchmarks too. Berkeley's rule transfers directly to the Design Lab: run a do-nothing agent against your detection matrix. If it scores above zero, your test has a bug. (Reliability over repeated runs: T4.)

OpenAI, "Why SWE-bench Verified no longer measures frontier coding capabilities" (Feb 2026); Berkeley RDI, "How We Broke Top AI Agent Benchmarks" (Apr 2026). Both checked 12 Sep 2026.

Limits and Judgment

What the workflow changes, and what remains human

Seven Tests for Any AI-Assisted Research Output

Before a script, figure, or table an agent produced goes into your paper, put it through seven questions. Any “no” or “I don’t know” means it is not yet research-grade.

Test	The question
1. Correctness	Did it compute, extract, or summarize the right object — and can you re-check the math, data flow, and magnitudes independently?
2. Reproducibility	Can a co-author rerun it from a clean clone — pinned model, prompt, seed, and data version?
3. Auditability	Can a referee trace the commands, checks, and sources that produced it?
4. Privacy boundary	Which files stayed local, and what content may still have crossed the network?
5. Cost	What did it cost — in tokens and dollars, including what a replicator will pay?
6. Speed	Net of checking and fixing, did it actually save time?
7. Skill investment	Is your team building durable skills — or quietly losing them?

Distinct from T5a’s five plan-grading criteria, which judge a plan before any code exists; these judge the output.

Three Decisions You Cannot Delegate

Not because agents cannot talk about them — they can, fluently. Because the researcher bears the consequences, and a delegated decision still has your name on it.

- **Identification — which variation answers the causal question.**

An agent can describe instruments, event windows, and shocks, and sometimes spot a weak one. It cannot *certify* that a shock is exogenous in your setting; that claim is yours to defend in a seminar.

- **Welfare — which objective defines “good.”**

Pareto weights, discount rates, the equity–efficiency trade-off: normative choices, not technical ones.

- **Judgment — which question is worth asking.**

An agent optimizes the target it is given; deciding which target is interesting, novel, and policy-relevant is the job.

Capability is not the issue. HA-Ramsey’s AI iteration surfaced real methodological discoveries (Lec10 T8), and Lec10 T9’s assistant passed every gate for the wrong reason. In both, the economist decided what counted. That is the residual claim a principal keeps.

When NOT to Use an Agent

The seven tests also mark a boundary: some research tasks cost more to delegate than they return, or fail too expensively.

- **One-line scripts and isolated edits.** A two-line sed or one-shot plot has no validation surface for the harness to add value. Open the editor.
- **Deeply proprietary or undocumented pipelines.** If the conventions live in one person's head, the agent has nothing to read; the time you would spend documenting the pipeline well enough to brief the agent is the same time you would spend just fixing the thing.
- **Strict deterministic guarantees.** For numerical reproducibility audits, regulatory pipelines, or replication packages, model nondeterminism is a hazard the harness cannot fully suppress. Run it pinned; do not let the agent retry.
- **Restricted-data analyses where prompts may leak.** If pasting one column would violate IRB or HIPAA, the agent does not belong in the loop; the red-lines workarounds apply.
- **Tasks where you do not yet have a benchmark.** Without a way to validate, faster output just means faster wrong. Define the V0 benchmark first; *then* bring in the agent.

Practical rule: the agent is most useful when the work is bounded, validated, and repeatable. None of those? Skip the agent.

Trust but Verify

1. Reproduce one statistic by hand
2. Cross-check one result with another package or script
3. Test a limiting case or zero-variance benchmark
4. Read the code line by line where the economics enters
5. Ask whether the magnitude and sign make sense

This is not optional ceremony. It is what turns fast code into defensible research.

What Changes for Economic Research

- The distance from question to first result shrinks
- More specifications become cheap enough to try
- Reproducibility can improve because workflow is written down
- The premium shifts away from typing speed and toward design, judgment, and validation

What stays human: asking the question, choosing the design, knowing the data, and interpreting the output.

The Paradigm Shift, Completed: What Changed, What Did Not

The HA-Ramsey record, read one last time — now as a statement about your role.

What changed

- Theory–code–debug loop compressed from months to days
- Documentation became executable: the TeX note, pseudocode, and code stay synchronized
- Cross-literature imports got cheap (computational geometry, viability theory)
- The unit of progress became the *validated milestone*

What did NOT change:

- AI never decided what was economically interesting
- AI never caught the wrong timing on its own
- AI never knew when a result was not credible

“Weeks, not years — and the binding constraint was economist judgment at the milestones, not implementation labor.”

Read the two columns together: the machine moved every bound except the one that defines the researcher.

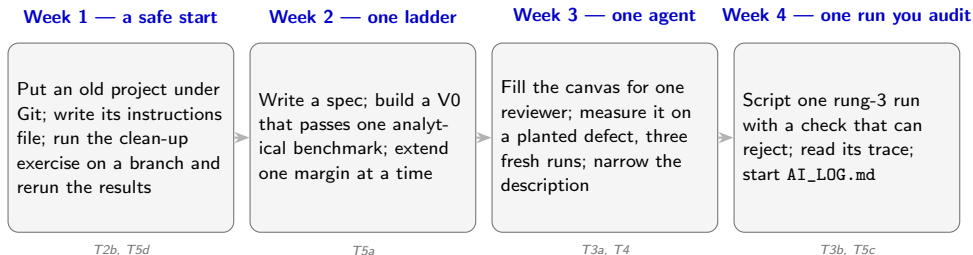
Source: HA-Ramsey case-study package (Lec10 T8).

Action Plan

Your first thirty days

Your First 30 Days

Four weeks, one habit each, on a project you already have. By the end you have the scaffold the capstone's first two milestones ask for.



Capstone: weeks 1–2 are M2's “repo scaffolded, benchmark in hand”; the log you start in week 4 is the graded `AI_LOG.md`. Works the same in Claude Code, Codex CLI, or Gemini CLI.

Your First Exercise: Clean Up Old Code

The best first exercise is not “do research for me.” It is: clean up one of your own old projects — safely, on a branch, and prove nothing changed.

```
I want your help cleaning up this project and setting up a git repo.  
First, copy it all over to a backup directory.  
Next, suggest an initial gitignore and create a first repo.  
Then, push the initial repo to a private repo on GitHub.  
Then, check out a new branch and suggest how to restructure the repo  
to have best practices with data/code/docs/paper organization.  
Then just do it on the new branch.  
Finally, actually run any code you've modified/restructured to make  
sure it lines up with the prior results.
```

Why each line is there

- **backup directory:** you can always go back
- **gitignore:** code apart from data and output
- **private repository:** history off your machine
- **new branch:** the restructuring is reversible
- **rerun and compare:** the check that the clean-up changed the layout, not the results

Prompt adapted from Aniket Panjwani, "AI Agents for Economics Research" (CEPR/VoxDev webinar, 2026). Private data stays out of the repository: T5d's red lines apply.

First Exercise and Looking Ahead

Best first exercise: take one old project and ask an agent to help clean it up safely — the prompt is on the previous frame.

- put the project under Git
- create an instructions file (`CLAUDE.md`, `AGENTS.md`, or `GEMINI.md`)
- identify one repeated workflow worth encoding
- rerun the cleaned project and compare outputs

Lec. 10: we return to case studies and let students run a full workflow themselves.

Setup details for each harness: Appendix TA.

Next — Lecture 10

From workflow logic to concrete case studies

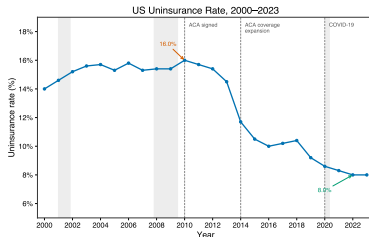
Lecture 10 Preview: Four Workflow Patterns, Seven Cases

Pattern	Lecture 10 case	What students will see
Reusable skill	Paper-review workflow	staged quality control, quote checks, editorial filter
Empirical pipeline	MEPS uninsurance	harmonization, survey weights, benchmark validation, figures
Public-web dataset	Fellows metadata	source restrictions, provenance, missingness discipline
Full-lifecycle collaborator	Glue-work project; HA-Ramsey	theory, pipeline design, critique, claims discipline
<i>Cautionary case</i>	MPE shortcut (Case 7)	a solver that passed every gate for the wrong reason

Why this matters: Lec. 9 gives the workflow logic. Lec. 10 shows how the same logic changes shape across different research artifacts.

One Visual Hook from Lecture 10: MEPS Uninsurance

Case Study 2 asks: who is uninsured in the United States, 2000–2023, and how did the ACA change the distribution?



Why keep this figure here: it is an accessible policy example for economists outside macro, and it shows what a validated empirical artifact looks like after the workflow succeeds.

Lec. 10 adds the full pipeline: prompts, schema harmonization, survey weights, external benchmarks, and subgroup figures.

What Role Does the Agent's Output Play in Your Inference?

The same toolkit produces different kinds of evidence. Ask what the agent's output is in your argument — that decides what you must check.

Pattern	The agent produces	Its role in your argument	What you must check
Reusable skill (paper review)	a structured critique	an input to your judgment, never the verdict	every quote against the paper
Empirical pipeline (MEPS)	a harmonized panel	the measurement itself	weights, crosswalks, published benchmarks
Public-web dataset (Fellows)	coded fields	a measurement with provenance	a source per field; missing stays missing
Full-lifecycle collaborator	code and drafts across a project	the whole chain of evidence	a gate at every milestone; the contract

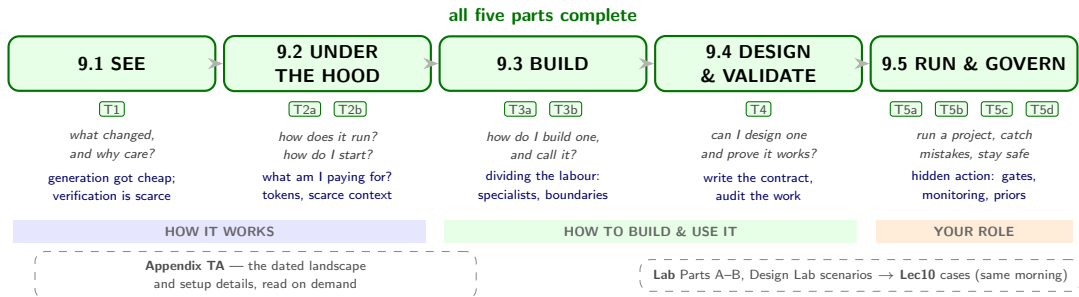
What to Watch for in Lecture 10

- **Paper review:** how a reusable skill turns a vague prompt into a staged quality-control workflow
- **MEPS pipeline:** how an empirical question becomes a validated panel and figure set
- **Fellows dataset:** how provenance and missingness rules matter as much as code
- **Glue-work project:** how agentic AI can act as a full-lifecycle collaborator without replacing human claims discipline

Bridge from Lec. 9 to Lec. 10: today's lecture explains the logic of the workflow; next lecture shows the artifacts it can produce.

Your Job Description: PI and Project Manager

The lecture's three questions, answered: *how it works* — a tool-calling loop steered by files (Parts 9.1–9.2); *how to build and use it* — agents you design and measure (Parts 9.3–9.4); *your role* — a method, a managed project, and rules (Part 9.5).



Agents can lay bricks quickly. You still design the structure. Set the goal. Define the final product. Gate every intermediate good. Assign the work — sessions, agents, sub-agents. Cross-verify everything. And remain the client of the outside view: the one thing your agent cannot give you is a prior other than your own.

Bridge: Lecture 10 Shows the Artifacts

Lecture 9 explained the logic. Lecture 10 shows the artifacts — the same morning.

- **You saw the Fellows workflow** — Lec10 T4 shows the findings, and Exercise B gives you a batch of your own
- **You saw HA-Ramsey's contract and milestones** — Lec10 T8 shows the discoveries and the economics
- **You saw the principal-agent lens** — Lec10 T9 shows it biting: an assistant that followed every rule it was given

“The agent didn't break the rules — it followed them. The loophole was in the contract.” Everything in Lecture 9 — the brief, the gates, the matrix, the log — is how you write a better contract, and how you check it was kept.